EconoCausal

Dynamic Pricing via Double Machine Learning

Progress report: Week 1 and Week 2

Project	Project 2: EconoCausal (Infotact Solutions, Advanced Data Science & ML Engineering, Vol. II)
Goal	Estimate how much a discount changes each customer's chance of buying, then spend a fixed budget on the customers it actually moves.
Stack	Python, DoWhy, EconML, scikit-learn, SciPy (Week 3), React, Plotly
Covered	Week 1 (causal graphing, data, propensity balancing, dashboard scaffold) and Week 2 (Double ML, uplift curves, refutation tests)

Why causal and not predictive

A standard model that predicts who is likely to buy will pick customers who would have bought anyway, so the discount is wasted on them. EconoCausal instead estimates the Individual Treatment Effect (ITE): the change in one customer's purchase probability caused by the discount. The budget then goes to the "Persuadables", the customers whose behaviour the discount actually changes.

Status

Week	Backend	Frontend	Status
1	Causal DAG, synthetic data, propensity balancing	Upload and budget views	Done
2	Double ML training (ITE), uplift evaluation, refutation tests	Uplift results tab (Plotly)	Done
3	SciPy budget optimizer	Allocation table	Planned
4	REST API, data drift detector	Polish, plain-language summaries	Planned

Part 1: Week 1

Causal graphing and dashboard scaffolding

Deliverables

- Mock retail dataset with confounders, treatment and outcome (`backend/data/generate_data.py`)
- Causal DAG defined in DoWhy (`backend/causal/build_dag.py`)
- Effect identification by backdoor adjustment (`docs/identified_estimand.txt`)
- Propensity score balancing with overlap and balance plots (`backend/causal/propensity.py`)
- React app with campaign upload and budget views (`frontend/`)

Data and causal setup

The dataset has 5,000 synthetic customers. Because it is simulated, the true effect of the discount on every customer is known, which lets us check later whether the model recovers it.

Role	Variables
Treatment	discount_given
Outcome	purchased
Confounders	past_spend, tenure_months, visits_per_month, price_sensitivity, region

Marketing historically discounted loyal, high-spending customers, so treated and untreated customers differ before any discount is applied. This is confounding: a simple comparison mixes the effect of the discount with the effect of who was targeted. The DAG records this, and adjusting for the confounders removes the bias.

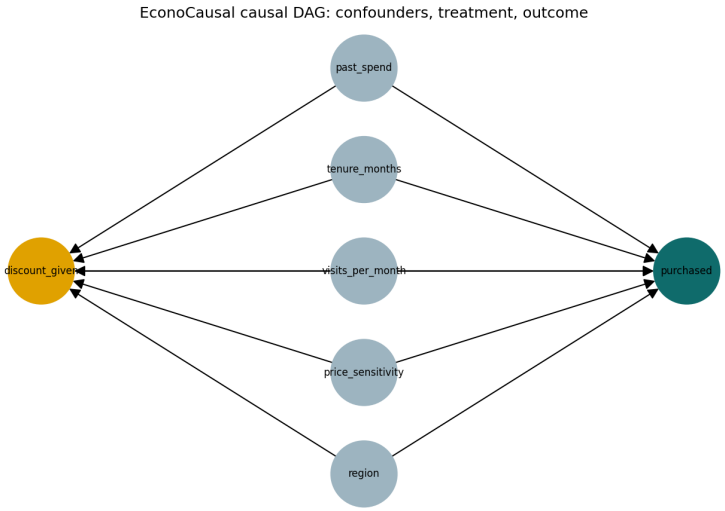


Figure 1. Causal DAG. Every confounder affects both the discount decision and the purchase.

Propensity score balancing

A logistic regression estimates each customer's probability of receiving the discount. Weighting customers by the inverse of that probability makes the two groups comparable, as if the discount had been assigned at random. A covariate is considered balanced when its standardised mean difference (SMD) is below 0.1.

Covariate	SMD before weighting	SMD after weighting
past_spend	+0.748	+0.015
tenure_months	+0.399	-0.016
visits_per_month	+0.289	-0.012
price_sensitivity	-0.004	+0.006
region	+0.039	+0.015

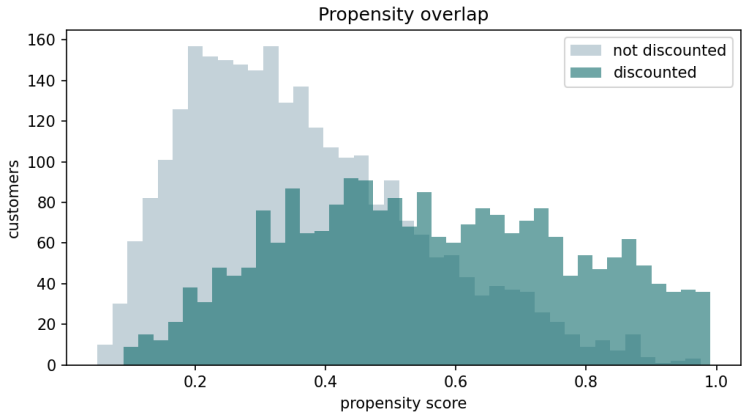


Figure 2. Propensity overlap. Both groups cover the same range, so comparison is valid.

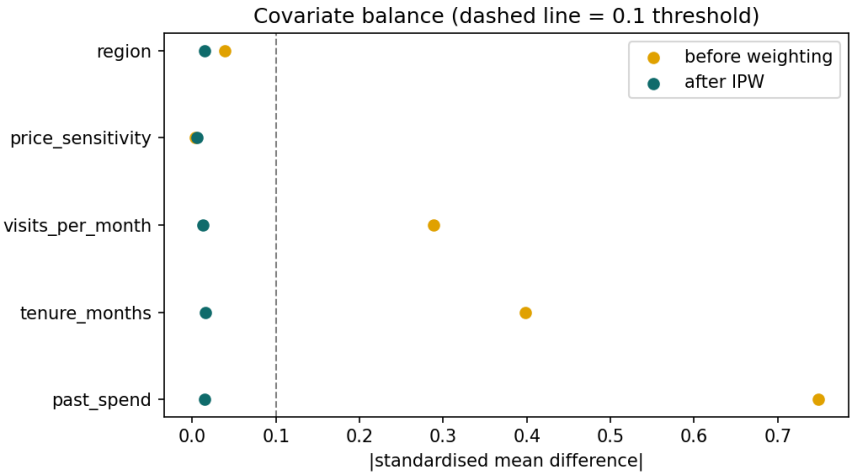


Figure 3. Covariate balance before and after weighting. All covariates fall under the 0.1 line after weighting.

Week 1 results

Estimate of the average discount effect	Value
Naive difference in purchase rate	+0.280
Linear backdoor adjustment	+0.161
Propensity (IPW) adjusted	+0.154
True effect (known from simulation)	+0.125

The naive number is more than double the truth because of the biased targeting. Adjustment brings it much closer. The remaining gap exists because the true effect varies from customer to customer and the relationships are not linear, which is the reason for Double Machine Learning in Week 2.

Dashboard (Week 1)

The React app has two views. The upload view accepts a campaign CSV, checks the required columns (customer_id, discount_given, purchased), and shows a preview and summary statistics. The budget view sets the total discount budget, the discount options and the per-customer cap.

Part 2: Week 2

Double Machine Learning and uplift curves

Deliverables

- Double ML with EconML: LinearDML and CausalForestDML (`backend/causal/train_dml.py`)
- Per-customer effects for 1,500 held-out customers (`backend/data/ite_scores.csv`)
- Qini curves and decile analysis (`backend/causal/uplift.py`)
- Predictive baseline to compare against ordinary machine learning
- DoWhy refutation tests for the Mid Review Causal Audit (`backend/causal/refute.py`)
- React Uplift results tab with Plotly charts (`frontend/src/components/UpliftView.jsx`)

Method

Double ML fits one model that predicts the outcome from the confounders and one that predicts the treatment from the same confounders, using 5-fold cross-fitting so no customer is scored by a model that saw them. It removes what the confounders explain from both, then estimates the effect of the discount from what remains. The causal forest lets that effect vary non-linearly across customers. Random Forest base learners are used. The data is split 70/30, stratified by treatment, and every result below is on the held-out 30%.

Results

Measure	Value
True average effect (simulation)	+0.127
LinearDML average effect	+0.146 (95% CI +0.112 to +0.180)
CausalForestDML average effect	+0.150
Correlation with true per-customer effect	linear 0.86, forest 0.80, predictive baseline -0.28
RMSE vs true per-customer effect	linear 0.056, forest 0.057

The confidence interval for the average effect contains the true value. The predictive baseline is negatively correlated with the true effect: customers who are likely to buy are the ones the discount helps least.

Extra purchases from discounting the best-ranked 30% of customers (ground truth):

Targeting method	Extra purchases
Oracle (ranked by the true effect)	101.8
Linear DML	96.9
Causal forest (DML)	93.6
Random rollout	57.0
Predictive baseline (likely buyers)	44.0

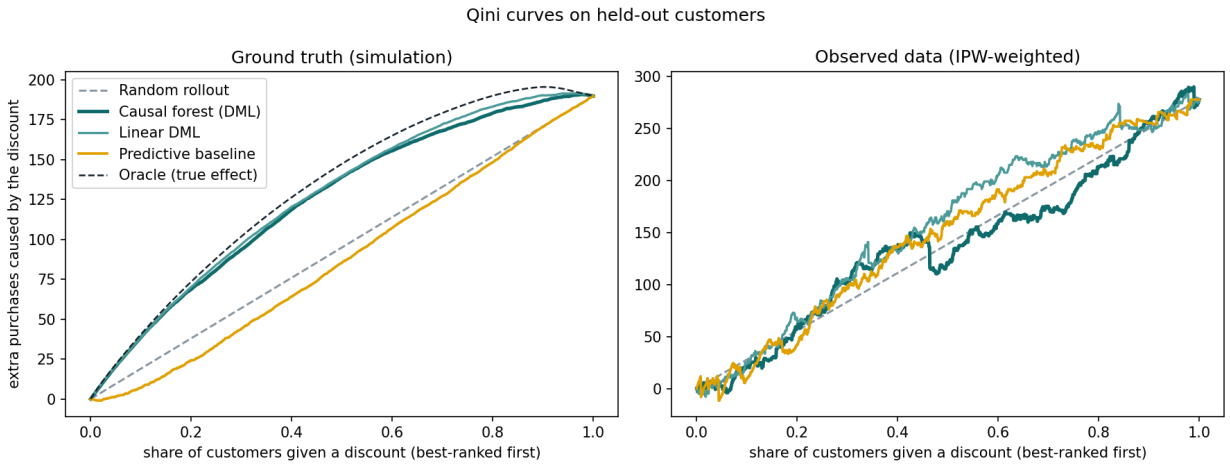


Figure 4. Qini curves. Left: ground truth from the simulation. Right: observed data only, weighted by propensity (noisier). A line above the dashed random line beats guessing.

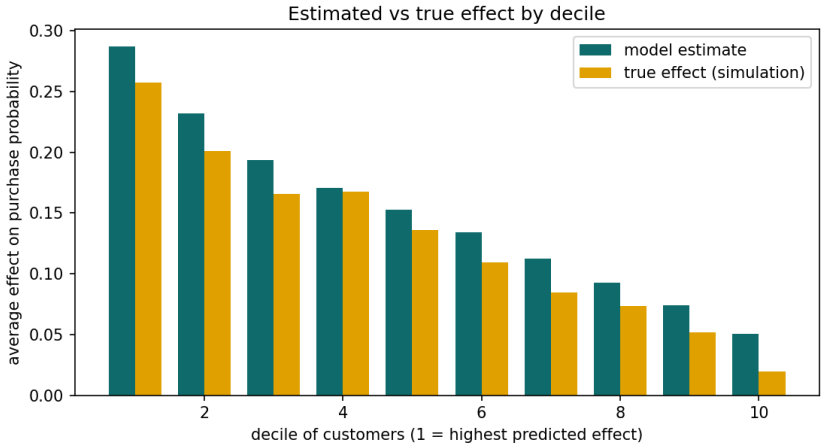


Figure 5. Customers sorted by predicted effect and split into ten groups. The model estimate tracks the true effect and falls from the first group to the last.

Causal audit: refutation tests

Refutation tests try to break the estimate. If it survives, that is evidence the estimate reflects a real causal effect.

Test	Original effect	New effect	Reading
Placebo treatment	+0.161	-0.001	Effect disappears with a fake treatment
Random common cause	+0.161	+0.161	Unchanged by an irrelevant confounder
Data subset (80%)	+0.161	+0.160	Stable across random subsets

Limitations

- The linear model matches or slightly beats the forest here because the simulated effect is close to linear. The forest should do better on data with more complex effects.
- The observed-data Qini curve is noisy at 1,500 customers. The forest beats random in the top 30% there but not clearly over the full curve, so the ground-truth curve is the cleaner proof.
- The data is simulated. Real campaign data will be harder to interpret.

Next: Week 3

A SciPy optimizer will allocate a fixed budget across customers using these effects, and the dashboard will show the final allocation table.